Analiza Dużych Zbiorów Danych

Wykład 3: Przechowywanie danych: Formaty, Data Lake i Lakehouse

dr inż. Krzysztof Ropiak
Uniwersytet Warmińsko-Mazurski w Olsztynie
Wydział Matematyki i Informatyki

Plan wykładu

- **1. Ewolucja paradygmatów składowania i fizyka I/O**
  Ewolucja nośników: DWH vs Data Lake vs Modern Lakehouse • Granice skalowania RDBMS • Rola baz NoSQL jako warstwy servingowej • Układ wierszowy vs kolumnowy • Sprzęt: CPU cache L1/L2/L3 i SIMD
- **2. Realia produkcyjne: Studium FinOps i techniki kodowania**
  Studium e-commerce "ShopGlobal" (Black Friday) • Anatomia zapytania analitycznego • Koszt skanowania 1 TB w S3/Athena (CSV vs Parquet) • Kodowanie słownikowe, RLE i Bit-Packing • Kompresja Snappy/ZSTD • Apache Arrow
- **3. Nowoczesne rozwiązanie: Anatomia Parquet i formaty tabelaryczne ACID**
  Struktura pliku Parquet (Row Groups, Column Chunks, Pages) • Column Projection i Predicate Pushdown • Dlaczego sam Parquet na S3 to NIE baza danych? • Problem braku ACID i listowania • Delta Lake (`_delta_log`, OCC) • Apache Iceberg (drzewo metadanych, Hidden Partitioning)
- **4. Zaawansowane mechanizmy Lakehouse, synteza i dyskusja**
  Time Travel i audytowalność • Modyfikacje danych: Copy-on-Write (CoW) vs Merge-on-Read (MoR) • Higiena plików i Z-Ordering • Macierz Delta vs Iceberg vs Hudi • Studia decyzyjne dla sali

01. Ewolucja nośników danych: Trzy generacje platform

W ciągu ostatnich 30 lat paradygmat składowania danych analitycznych przeszedł trzykrotną rewolucję:

I Generacja: DWH

Lata 1990–2010

- Relacyjne silniki SQL (Oracle, Teradata)
- Ścisły schemat *Schema-on-Write*
- Tylko dane ustrukturyzowane
- Wysoki koszt macierzy SAN/dysków
- Brak wsparcia dla Data Science / ML

➔

II Generacja: Data Lake

Lata 2010–2020

- HDFS, a następnie chmurowy S3 / ADLS
- Podejście *Schema-on-Read*
- Dowolne pliki: JSON, CSV, Parquet, audio
- Bardzo tani storage, separacja Compute
- **Brak ACID ➔ chaos i Data Swamp**

➔

III Generacja: Lakehouse

Od 2020 roku

- Tani Object Storage jako nośnik fizyczny
- Formaty tabelaryczne: Delta Lake, Iceberg
- **Pełna transakcyjność ACID na plikach**
- Wydajność SQL DWH + elastyczność ML
- Jedno źródło prawdy (Single Source of Truth)

Kluczowa synteza: Lakehouse nie jest "kolejnym nowym nośnikiem fizycznym". To **nakładka logiczna i transakcyjna** (zarządzana warstwa metadanych) nałożona na tanie pliki w obiektowej chmurze.

Źródło: M. Armbrust, A. Ghodsi, M. Zaharia et al., "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics", CIDR 2021.

02. Granice skalowalności RDBMS: Kiedy baza relacyjna zawodzi?

Tradycyjne bazy relacyjne (PostgreSQL, MySQL, Oracle) zostały zoptymalizowane pod transakcje **OLTP** (*Online Transaction Processing*), a nie analitykę **OLAP** (*Online Analytical Processing*):

Wzorzec OLTP (Transakcyjny)

Bankomaty, e-commerce, logowanie, koszyki

- Zapytania czytają lub modyfikują **pojedyncze rekordy** (np. `SELECT * WHERE user_id = 42`).
- Wymóg submilisekundowej latencji i twardej izolacji transakcji ACID.
- Układ wierszowy: cały rekord leży w jednym ciągłym bloku pamięci na dysku.
- Indeksy B-Tree idealnie wskazują pozycję wiersza.

Wzorzec OLAP (Analityczny)

Raporty zarządcze, trendy roczne, agregacje, ML

- Zapytania skanują **miliardy wierszy** (np. `SUM(kwota) GROUP BY kategoria`).
- Odczytywane są tylko **2–3 kolumny** spośród 100 dostępnych w tabeli.
- W bazie wierszowej dysk i magistrala muszą przetransportować 100% niepotrzebnych danych!
- **Indeksy B-Tree są bezużyteczne przy skanowaniu całej tabeli.**

Wąskie gardło I/O: Jeśli tabela ma 100 kolumn, a analityk sumuje jedną, silnik wierszowy RDBMS marnuje **99% przepustowości dysku i magistrali RAM** na odczytywanie i odrzucanie niechcianych atrybutów.

Źródło: D. J. Abadi et al., "Column-Stores vs. Row-Stores: How Different Are They Really?", ACM SIGMOD 2008.

03. NoSQL w architekturze danych: Warstwa Servingowa

Jeśli formaty kolumnowe królują w analityce, dlaczego w architekturach Big Data wciąż powszechnie stosujemy bazy **NoSQL**?

Key-Value / Document

Redis, AWS DynamoDB, MongoDB

- Dostęp przez klucz główny (hash lookup).
- Latencja: **< 2–5 ms**.
- Zastosowanie: profile użytkowników, sesje, koszyki w czasie rzeczywistym.

Wide-Column Stores

Apache Cassandra, Apache HBase, ScyllaDB

- Liniowa skalowalność zapisu (LSM-trees / SSTables).
- Miliony operacji zapisu/s z telemetrii IoT.
- Zastosowanie: time-series, historia sensorów, licznik zdarzeń.

Search & Real-time OLAP

Elasticsearch, ClickHouse, Apache Pinot

- Indeksy odwrócone (Inverted Index) lub ultrawydajny wektorowy OLAP.
- Latencja zapytań: **50–200 ms**.
- Zastosowanie: dashboardy dla milionów użytkowników (User-Facing Analytics).

Rola w nowoczesnym potoku: Bazy NoSQL nie zastępują Lakehouse'a – pełnią funkcję **wyspecjalizowanej warstwy dostępu (Serving Layer)**. Po przeliczeniu zagregowanych metryk w Sparku zrzucamy wyniki do DynamoDB lub ClickHouse, aby aplikacja webowa mogła je serwować milionom klientów.

Źródło: M. Kleppmann, "Designing Data-Intensive Applications", O'Reilly 2017 (Rozdział 3).

04. Fizyka nośników: Układ wierszowy vs kolumnowy

Wyobraźmy sobie tabelę transakcji: `ID (int)`, `Data (date)`, `Klient (str)`, `Kwota (float)`:

```
Układ wierszowy (Row-Oriented / CSV / RDBMS):
[ID_1, Data_1, Klient_1, Kwota_1] [ID_2, Data_2, Klient_2, Kwota_2] [ID_3, Data_3, Klient_3, Kwota_3] ...
                                          ▲
      Aby odczytać tylko kwoty, głowica dysku / kontroler pamięci musi przeskanować wszystkie bajty!

Układ kolumnowy (Column-Oriented / Parquet / ORC):
[ID_1, ID_2, ID_3...]  [Data_1, Data_2, Data_3...]  [Klient_1, Klient_2...]  [Kwota_1, Kwota_2, Kwota_3...]
                                                                                      ▲
             Silnik analityczny ładuje z nośnika TYLKO ten ciągły blok bajtów!
```

Zapis wierszowy (Row Layout)

- **Zaleta:** Błyskawiczny zapis i odczyt całego rekordu (jeden ciągły strumień I/O).
- **Wada analityczna:** Ogromny narzut transferu niepotrzebnych atrybutów.
- **Kompresja:** Niska (różne typy danych leżą obok siebie: int, string, float).

Zapis kolumnowy (Column Layout)

- **Zaleta:** Odczyt ściśle tych kolumn, które występują w zapytaniu (*Column Projection*).
- **Zaleta:** Identyczne typy danych leżą obok siebie ➔ **wybitny stopień kompresji**.
- **Wada:** Zapis pojedynczego wiersza wymaga modyfikacji wielu bloków dyskowych.

Źródło: M. Stonebraker et al., "C-Store: A Column-oriented DBMS", VLDB 2005.

05. Sprzętowy fundament wydajności: Cache CPU i wektoryzacja SIMD

Dlaczego format kolumnowy działa w silnikach analitycznych nawet **100x szybciej** niż wierszowy? Sekret tkwi w mikroskopowej architekturze procesora:

Pamięć podręczna CPU (L1/L2/L3 Cache)

- Dane z pamięci RAM do rdzenia CPU wędrują w tzw. **liniach cache (Cache Lines)** o rozmiarze 64 bajtów.
- W modelu wierszowym 64-bajtowa linia zawiera 1 liczbę (kwotę) oraz śmieci (imię, nazwisko, adres).
- W modelu kolumnowym 64 bajty zawierają **8 kolejnych 64-bitowych liczb typu float** – 100% użyteczności cache!
- Sprzętowy *Hardware Prefetcher* procesora z łatwością przewiduje sekwencyjny odczyt kolumny.

Instrukcje wektorowe SIMD

- **SIMD** (*Single Instruction, Multiple Data*) – np. instrukcje AVX-512 / ARM NEON.
- Procesor wykonuje operację arytmetyczną (np. dodawanie kwot) na **8 lub 16 liczbach w JEDNYM takcie zegara**.
- Wektorowe przetwarzanie wymaga, aby liczby leżały w pamięci ciągle i bez przerw.
- Format kolumnowy umożliwia kompilatorom i silnikom (DuckDB, Photon, ClickHouse) pełną wektoryzację pętli obliczeniowych.

Reguła inżynierska: Nowoczesna analityka Big Data nie jest ograniczona taktowaniem procesora, lecz **przepustowością magistrali pamięci (Memory Bandwidth)** i chybieniami pamięci podręcznej (*Cache Misses*). Układ kolumnowy eliminuje to wąskie gardło.

Źródło: P. Boncz, M. Zukowski, N. Nes, "MonetDB/X100: Hyper-Pipelining Query Execution", CIDR 2005.

06. Studium przypadku: E-Commerce "ShopGlobal" po Black Friday

Scenariusz wdrożeniowy (Produkcja):
Platforma "ShopGlobal" zarejestrowała podczas weekendu Black Friday **500 milionów zdarzeń użytkowników** (kliki, dodania do koszyka, zakupy). Zespół Data Engineering zapisał wszystkie surowe logi w chmurze AWS S3 w standardowym formacie **pliku tekstowego CSV**.

- **Parametry zbioru danych:**
  - Łączna liczba zdarzeń: 500 000 000 wierszy.
  - Liczba atrybutów w zdarzeniu: **40 kolumn** (nagłówki HTTP, cookies, payload JSON, adres IP, geolokalizacja, identyfikatory sesji).
  - Całkowity rozmiar danych na S3 w CSV: **1 000 GB (1 Terabajt)**.
- **Zadanie od Dyrektora Finansowego w poniedziałek rano:**
  > *"Podajcie mi łączną wartość sprzedaży z podziałem na kraje w niedzielę Black Friday."*
- **Zapytanie analityka w SQL:**

```sql
SELECT country_code, SUM(transaction_amount) FROM shopglobal_events
WHERE event_date = '2026-11-29' AND event_type = 'PURCHASE' GROUP BY country_code;
```

- **Kluczowa obserwacja:** Zapytanie dotyczy tylko **4 kolumn** (`country_code`, `transaction_amount`, `event_date`, `event_type`) z 40 dostępnych! Pozostałe 36 kolumn jest całkowicie ignorowanych.

07. Anatomia zapytania analitycznego: Co tak naprawdę dzieje się na S3?

Kiedy silnik SQL (np. AWS Athena / Presto / Trino) wykonuje to zapytanie na surowych plikach CSV:

```
Zapytanie potrzebuje: [country_code] [transaction_amount] [event_date] [event_type] (10% objętości rekordu)
                                          |
                                          ▼
          CO ROBI SILNIK SQL PRZY FORMACIE TEKSTOWYM (CSV / JSON):
┌──────────────────────────────────────────────────────────────────────────┐
│ 1. Skanuje cały 1 TB danych przez sieć z AWS S3 do węzłów klastra obliczeniowego │
│ 2. Zużywa setki cykli CPU na parsowanie stringów, szukanie przecinków i '\n'    │
│ 3. Konwertuje tekstowe "149.99" na 64-bitowy float w pamięci RAM                │
│ 4. Wyrzuca do śmieci 90% sparsowanych danych (pozostałe 36 kolumn)              │
│ 5. Odpala filtr WHERE po rozpakowaniu każdego wiersza                           │
└──────────────────────────────────────────────────────────────────────────┘
```

Dlaczego format CSV jest katastrofą w Big Data? 1. **Brak metadanych o typach:** Każda cyfra to ciąg znaków ASCII (np. "1000000" zajmuje 7 bajtów zamiast 4 bajtów int32). 2. **Brak możliwości skakania:** Nie można przeczytać 10. kolumny bez sekwencyjnego sparsowania pierwszych 9 kolumn i policzenia przecinków! 3. **Brak statystyk min/max:** Aby sprawdzić warunek `WHERE event_date = '2026-11-29'`, trzeba przeczytać każdy plik od pierwszej do ostatniej linijki.

Źródło: AWS Big Data Architecture Blog, "Optimizing Amazon Athena Queries", 2021.

08. FinOps w chmurze: Koszt skanowania 1 TB w AWS Athena

Model rozliczeniowy usług serverless SQL w chmurze (AWS Athena, Google BigQuery) jest bezlitosny: **płacisz za każdy bajt odczytany ze storage'u**.

Wariant A: Pliki tekstowe CSV

- Ilość przeskanowanych danych: **1 000 GB (1,00 TB)**
- Cennik AWS Athena: $5,00 za każdy przeskanowany TB
- Koszt pojedynczego zapytania: **5,00 USD**
- Czas trwania zapytania: **240 sekund (4 minuty)**
- Dashboard odświeża się 100 razy dziennie:
  100 × $5,00 = $500 dziennie = $15 000 / miesięcznie!

Wariant B: Kolumnowy Apache Parquet

- Dzięki kompresji Snappy rozmiar danych spada z 1 TB do **130 GB**.
- Dzięki *Column Projection* czytamy tylko 4 kolumny: **~25 GB**.
- Dzięki *Predicate Pushdown* (filtr daty) czytamy tylko: **~5 GB**!
- Koszt pojedynczego zapytania: **0,025 USD (2,5 centa)**
- Czas trwania: **3 sekundy** (80x szybciej!)
- Miesięczny koszt: **75 USD (oszczędność 99,5%!)**

Lekcja FinOps: Wybór formatu fizycznego pliku w chmurze to nie detal implementacyjny. To różnica pomiędzy rachunkiem na **180 000 USD rocznie** a rachunkiem na **900 USD rocznie** za ten sam dashboard analityczny!

Źródło: AWS Documentation, "Amazon Athena Pricing Model & Performance Optimization", Amazon Web Services, 2023.

09. Zestawienie empiryczne: CSV vs Gzip CSV vs Apache Parquet

Twarde dane z benchmarków inżynierskich na zbiorze 1 TB surowych logów:

Format pliku	Objętość na dysku S3	Czas wykonania zapytania	Dane przeskanowane (I/O)	Koszt zapytania (Athena)	Czy wspiera Column Projection?	Czy plik jest dzielony (Splittable)?
Surowy CSV	1 000 GB (1,00 TB)	~235 s	1 000 GB	**$5,00**	**NIE**	**TAK**
Gzipped CSV (.csv.gz)	220 GB	~190 s	220 GB	**$1,10**	**NIE**	**NIE (1 plik = 1 rdzeń!)**
Apache Parquet (Uncompressed)	350 GB	~18 s	35 GB	**$0,175**	**TAK**	**TAK**
Apache Parquet (Snappy)	130 GB	~4 s	13 GB	**$0,065**	**TAK**	**TAK**
Apache Parquet (ZSTD)	**95 GB**	~5 s	**9,5 GB**	**$0,047**	**TAK**	**TAK**

Pułapka Gzip CSV: Standardowa kompresja Gzip na plikach CSV tworzy **strumień niedzielony (Non-splittable)**. Oznacza to, że pojedynczy plik 10 GB skompresowany gzipem może być dekompresowany wyłącznie przez **jeden rdzeń CPU**! Klaster 100 maszyn Sparka stoi bezczynnie, czekając na jeden wątek.

10. Algorytmy kodowania kolumnowego: Encoding przed kompresją

W formacie kolumnowym dane o identycznym typie ułożone są obok siebie. Zanim uruchomimy kosztowny algorytm kompresji ogólnej, stosujemy sprytne **techniki kodowania (Encoding)**:

1. Dictionary Encoding

Kodowanie słownikowe

- Teksty (np. nazwy krajów: 'Polska', 'Niemcy') zamieniane są na liczby (0, 1, 2).
- Słownik zapisywany jest w nagłówku strony.
- Zamiast powtarzać 20-bajtowy string, zapisujemy 2-bajtowy indeks integer.

2. Run-Length Encoding (RLE)

Kodowanie długości serii

- Ciąg powtarzających się wartości: `PL, PL, PL, PL, PL`.
- Zapisywany jako para: `(5, PL)`.
- Genialna skuteczność dla danych posortowanych lub o niskiej kardynalności.

3. Bit-Packing & Delta Encoding

Upakowanie bitowe

- Jeśli liczby mieszczą się w zakresie 0–3, nie marnujemy 32 bitów – zapisujemy je na **2 bitach**.
- *Delta Encoding:* zapisujemy tylko różnice między kolejnymi wartościami (np. znaczniki czasu: 1000, 1002, 1005 ➔ 1000, +2, +3).

Efekt synergii: Dzięki kodowaniu kolumnowemu zbiór danych zmniejsza swoją objętość o **60–80% JESZCZE ZANIM** zostanie poddany kompresji blokowej (Snappy/ZSTD)!

Źródło: Apache Parquet Specification, "Encodings Overview", Apache Software Foundation.

11. Kompresja blokowa: Snappy vs ZSTD vs Gzip

Po zakodowaniu danych następuje etap kompresji blokowej stron. W inżynierii Big Data kompresja to nie walka o "najmniejszy plik", lecz **kompromis pomiędzy stopniem kompresji a szybkością dekompresji (CPU throughput)**:

Snappy (Google) – Domyślny standard w Big Data

Zaprojektowany z myślą o maksymalnej przepustowości. Dekompresja z prędkością > **500 MB/s na rdzeń CPU**. Nie marnuje cykli procesora. Umiarkowany stopień kompresji.

SZYBKOŚĆ: 10/10 | KOMPRESJA: 6/10

ZSTD (Zstandard – Facebook/Meta) – Nowy król analityki

Nowoczesny algorytm łączący znakomity stopień kompresji (zbliżony do Gzip) z zawrotną prędkością dekompresji. Konfigurowalne poziomy (1–22). Coraz częściej domyślny w Parquet i Iceberg.

SZYBKOŚĆ: 9/10 | KOMPRESJA: 9/10

Gzip / Deflate – Klasyk epoki internetowej

Wysoki stopień kompresji, ale **bardzo wolna dekompresja (wymaga dużej mocy CPU)**. W nowoczesnych klastrach analitycznych staje się wąskim gardłem obliczeniowym.

SZYBKOŚĆ: 3/10 | KOMPRESJA: 8/10

Wskazówka wdrożeniowa: W potokach analitycznych wybieraj **Snappy** dla danych roboczych (szybki zapis/odczyt) oraz **ZSTD** dla wielkich tabel historycznych (maksymalna redukcja kosztów S3 przy zachowaniu szybkiej dekompresji).

Źródło: Y. Collet, "Zstandard - Fast real-time compression algorithm", RFC 8878 / Meta Engineering, 2021.

12. Kolumny w pamięci RAM: Apache Arrow i Zero-Copy Interop

Parquet to standard przechowywania kolumnowego na dysku. Ale co dzieje się, gdy dane trafiają do pamięci operacyjnej (RAM)?

Tradycyjny chaos w pamięci RAM

Koszmar serializacji i konwersji typów

- Spark reprezentuje DataFrame w obiektach JVM.
- Python / Pandas wymaga tablic NumPy w pamięci C.
- Przekazanie danych ze Sparka do Pythona wymaga **serializacji do bajtów, transferu przez socket i deserializacji**.
- Narzut kopiowania i konwersji pożera do **70–80% czasu wykonania** skryptu ML!

Rewolucja Apache Arrow

Zunifikowany standard kolumnowy in-memory

- Wspólny format binarny w RAM dla C++, Pythona, R, Javy, Rusta i silników SQL (DuckDB, Spark).
- **Zero-Copy Sharing:** Procesy w różnych językach czytają te same bufory w pamięci współdzielonej bez kopiowania ani jednego bajtu!
- Bezpośrednie wsparcie dla wektoryzacji SIMD w RAM.

Podział ról w architekturze:

👉 **Apache Parquet** = zoptymalizowany pod kątem trwałego składowania na dysku (kompresja, kodowanie, mały rozmiar).

👉 **Apache Arrow** = zoptymalizowany pod kątem błyskawicznego przetwarzania w pamięci RAM (Zero-Copy, SIMD, brak serializacji).

Źródło: W. McKinney et al., "Apache Arrow: A Cross-Language Development Platform for In-Memory Analytics", ASF, 2016–2023.

13. Anatomia pliku Apache Parquet: Hierarchia wewnętrzna

Jak fizycznie zorganizowany jest plik `.parquet` na dysku lub w chmurze S3?

```
┌─────────────────────────────────────────────────────┐
│ Header: 4 bajty 'PAR1' (Magiczna sygnatura formatu) │
├─────────────────────────────────────────────────────┤
│ ROW GROUP 1 (np. 500 000 wierszy, typowo 128 MB - 512 MB)
│   ├── Column Chunk: ID          [Page 1] [Page 2] ...
│   ├── Column Chunk: Timestamp   [Page 1] [Page 2] ...
│   └── Column Chunk: Amount      [Data Page] [Dictionary Page]
├─────────────────────────────────────────────────────┤
│ ROW GROUP 2 (kolejne 500 000 wierszy)
│   ├── Column Chunk: ID          ...
│   ├── Column Chunk: Timestamp   ...
│   └── Column Chunk: Amount      ...
├─────────────────────────────────────────────────────┤
│ FILE METADATA FOOTER (Zapisywany na samym końcu pliku!):
│   ├── Schemat tabeli (kolumny, typy, definicje zagnieżdżeń)
│   ├── Wskaźniki offsetów (gdzie leży każdy Column Chunk i każda Page)
│   └── Statystyki per kolumna/grupa: MIN, MAX, NULL_COUNT
├─────────────────────────────────────────────────────┤
│ Footer Length (4 bajty) + Magiczna sygnatura 'PAR1' (4 bajty)
└─────────────────────────────────────────────────────┘
```

> **Dlaczego stopka leży na końcu?** Ponieważ podczas zapisu pliku streamingowego silnik nie zna statystyk (min/max) ani offsetów, dopóki nie zapisze wszystkich wierszy. Dlatego metadane zrzucane są na samym końcu pliku.

Źródło: Apache Parquet File Format Specification, Apache Software Foundation.

14. Omijanie danych: Column Projection i Predicate Pushdown

Dwa fundamentalne mechanizmy, dzięki którym silniki czytają tylko ułamek bajtów z pliku Parquet:

1. Projekcja kolumn (Column Projection)

- Silnik SQL analizuje zapytanie: `SELECT user_id, amount`.
- Czytnik pobiera z metadanych Footera **dokładne offsety bajtowe** tylko dla tych dwóch kolumn.
- Wysyła zapytania `HTTP GET Range` bezpośrednio po wskazane zakresy bajtów z S3.
- Kolumny nieużywane w zapytaniu **nigdy nie opuszczają dysków storage'u chmurowego**.

2. Wypychanie predykatów (Predicate Pushdown)

- Analityk pisze: `WHERE amount > 10000`.
- Czytnik sprawdza statystyki w Footerze dla Row Group 1: `min = 10, max = 500`.
- Ponieważ maksymalna wartość w tej grupie to 500, nie może w niej być rekordu > 10000.
- **Row Group 1 (np. 200 MB danych) jest całkowicie pomijana (Data Skipping)** bez pobierania jej z sieci!

Połączenie potęg: W zapytaniu analitycznym z filtrami łączymy selekcję kolumn z omijaniem grup wierszy. Często silnik pobiera zaledwie **1–2% fizycznych bajtów pliku**, zwracając wynik w ułamku sekundy.

Źródło: J. Le Dem (Co-creator of Parquet), "How Parquet Enables High-Performance Columnar Data", Twitter / Databricks.

15. Pułapka surowego Parquet na S3: To NIE jest baza danych!

Zachwyceni formatem Parquet inżynierowie zapisywali miliony plików w chmurze AWS S3 w strukturze katalogów Hive (`/year=2026/month=11/`). Dlaczego to rozwiązanie okazało się bombą zegarową?

Ograniczenia obiektowej pamięci S3

- **Brak atomowych operacji na katalogach:** Na S3 nie ma prawdziwych folderów – są tylko obiekty z ukośnikami w kluczach (`key`).
- Operacja zmiany nazwy folderu (`RENAME`) w S3 to fizyczne **skopiowanie każdego pliku obiekt po obiekcie i usunięcie starego**!
- Przy 10 000 plików operacja rename trwa kilkanaście minut i może się wywalić w połowie.

Brak spójności i wyścigi danych

- Jeśli proces Sparka zapisuje nową partycję (100 plików Parquet) i padnie przy 73. pliku:
  ➔ **Tabela zostaje ze śmieciami i danymi częściowymi!**
- Jeśli w tym samym czasie analityk odpala zapytanie:
  ➔ Czyta dane pół-zapisane, generując błędne raporty finansowe.
- Brak możliwości wykonania transakcyjnego `ROLLBACK` .

Wniosek architektoniczny: Sam plik Parquet rozwiązuje problem wydajności I/O pojedynczego pliku, ale **nie zapewnia żadnych gwarancji tabelarycznych**. Zbiór plików w katalogu to nie tabela!

Źródło: M. Armbrust et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores", VLDB 2020.

16. Koszmar operacji LIST i brak transakcji ACID na S3

Kolejny krytyczny problem klasycznych jezior danych opartych na surowych plikach Parquet:

Koszmar operacji listowania metadanych (Directory Listing)

- W tradycyjnym Hive/Sparku, aby dowiedzieć się, jakie pliki należą do tabeli, silnik wykonuje operację `LIST objects` w S3.
- API S3 zwraca maksymalnie **1 000 obiektów w jednej odpowiedzi**. Tabela z 1 000 000 plików wymaga 1 000 zapytań HTTP!
- Samo przygotowanie planu zapytania (zanim silnik przeczyta choćby 1 bajt danych) trwało w Netfliksie **10–40 minut** i kończyło się błędem `HTTP 503 Slow Down` z powodu przekroczenia limitów S3!

Brak izolacji transakcji ACID

- Dwa joby zapisujące dane do tej samej partycji nadpisują sobie wzajemnie pliki (*Write Conflict*).
- Brak atomowości: operacja *Insert-Overwrite* jest widoczna dla czytających jako "chwilowo pusta tabela".

Brak operacji UPDATE i DELETE

- Chcesz zaktualizować 1 wiersz na mocy RODO?
- W surowym Parquet musisz przeczytać cały zbiór, odnaleźć plik, przepisać go i zastąpić ręcznie na S3.

Źródło: R. Blue, D. Weeks, "Apache Iceberg: The Definitive Guide", O'Reilly 2023.

17. Narodziny formatów tabelarycznych: Nowa warstwa abstrakcji

Rozwiązaniem problemu stało się odseparowanie **pojęcia tabeli** od **struktury katalogów na dysku**:

Klasyczny Data Lake (Hive)

Tabela = Struktura katalogów

- Tabela to folder: `s3://bucket/tabela/`
- Partycje to podfoldery: `/rok=2026/miesiac=11/`
- Stan tabeli określany przez kosztowne listowanie plików na dysku.
- **Brak atomowości, podatność na awarie**.

➔

Nowoczesny Lakehouse (Formaty Tabelaryczne)

Tabela = Niezmienny log metadanych

- Stan tabeli definiowany przez **plik metadanych / log transakcyjny**.
- Tabela to ścisły zbiór konkretnych plików Parquet wskazanych w manifeście.
- **Operacje atomowe na poziomie metadanych (ACID).**
- Zero kosztownego listowania plików w S3!

Trzej liderzy rewolucji formatów tabelarycznych:

1. **Delta Lake:** Zaprojektowany przez twórców Apache Spark (Databricks), oparty na dzienniku transakcji `_delta_log`.
2. **Apache Iceberg:** Zaprojektowany przez inżynierów Netflixa i Apple, oparty na hierarchicznym drzewie manifestów.
3. **Apache Hudi:** Zaprojektowany przez Ubera, zoptymalizowany pod kątem streamingu i szybkich mutacji (Upsert).

Źródło: M. Armbrust et al. (CIDR 2021) / R. Blue et al. (O'Reilly 2023).

18. Delta Lake pod maską: Transakcyjny log `_delta_log`

Jak Delta Lake realizuje pełną transakcyjność ACID na tanich plikach w chmurze S3?

```
katalog_tabeli/
├── _delta_log/
│   ├── 00000000000000000000.json      ➔ Commit 0: Dodano pliki part-A.parquet, part-B.parquet
│   ├── 00000000000000000001.json      ➔ Commit 1: Dodano part-C.parquet, usunięto part-A.parquet
│   ├── 00000000000000000002.json      ➔ Commit 2: Dodano part-D.parquet
│   └── 00000000000000000010.checkpoint.parquet  ➔ Kompaktacja: Sumaryczny stan po 10 commitach!
├── part-A.parquet (stary plik, usunięty logicznie w Commit 1, fizycznie nadal obecny na dysku!)
├── part-B.parquet
├── part-C.parquet
└── part-D.parquet
```

Atomowy Commit (Pliki JSON)

- Każda transakcja (INSERT, DELETE, MERGE) zapisuje nowy plik JSON w folderze `_delta_log/`.
- Plik JSON wymienia operacje: `add(plik)` oraz `remove(plik)`.
- Operacja staje się widoczna dla świata **dokładnie w momencie udanego zapisu pliku JSON**.

Punkty kontrolne (Checkpoints)

- Co 10 commitów Delta generuje plik `checkpoint.parquet`.
- Czytnik nie musi parsować setek starych JSON-ów – czyta ostatni checkpoint + najnowsze delty.
- Błyskawiczny start zapytania bez dotykania plików danych.

Źródło: M. Armbrust, T. Das, L. Hsieh et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores", VLDB 2020.

19. Kontrola współbieżności: Protokół OCC w Delta Lake

Jak dwa niezależne procesy mogą jednocześnie zapisywać dane do tabeli bez używania ciężkich blokad bazodanowych (*Locks*)?

Optymistyczna kontrola współbieżności (Optimistic Concurrency Control – OCC)

Delta zakłada optymistycznie, że transakcje nie wejdą sobie w drogę. Wykrywa konflikty dopiero w momencie próby zatwierdzenia (Commit):

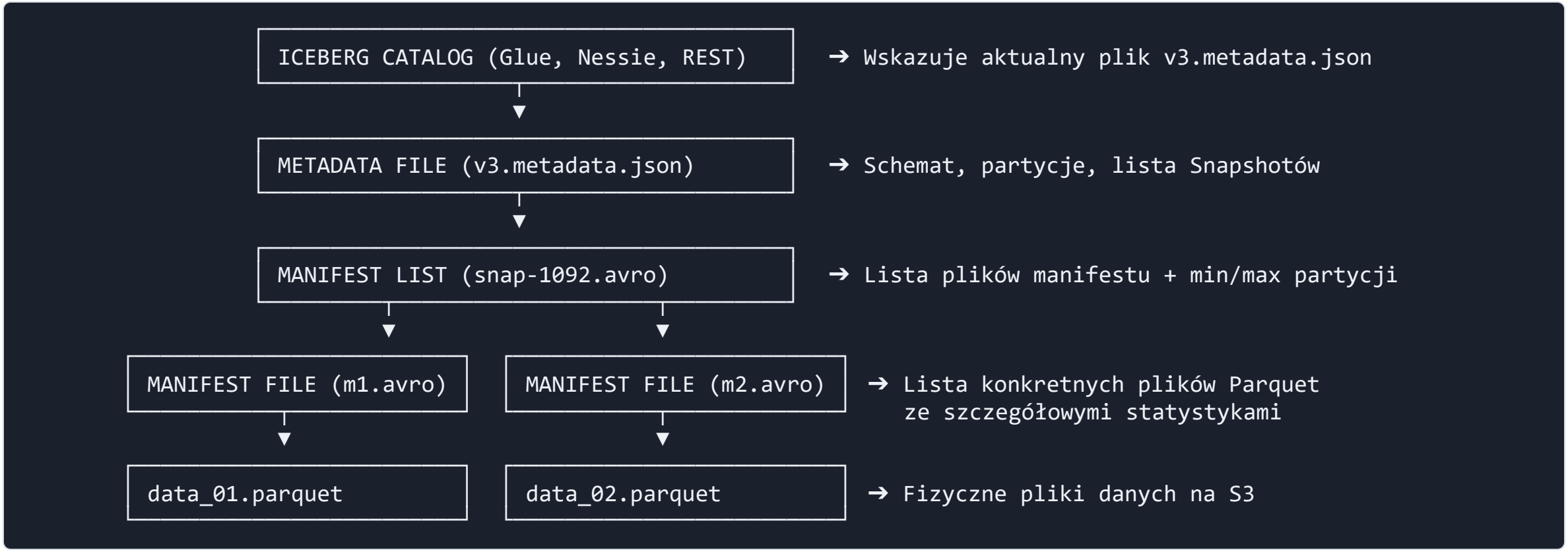
1. **Odczyt stanu:** Transakcja odczytuje aktualną wersję tabeli (np. Wersja 12).
2. **Wykonanie pracy:** Zapisuje nowe pliki danych Parquet na S3 (dla świata są one jeszcze niewidoczne).
3. **Próba zatwierdzenia:** Próbuje zapisać plik `000013.json`.
4. **Weryfikacja konfliktu:** Jeśli inny proces zapisał już wersję 13, Delta sprawdza: "*Czy wersja 13 zmieniła te same pliki, na których ja pracowałem?*".
   ➔ Jeśli NIE: Delta automatycznie ponawia próbę i zapisuje swoje zmiany jako `000014.json` (bez utraty pracy!).
   ➔ Jeśli TAK (konflikt zapisu): Transakcja rzuca wyjątek `ConcurrentModificationException`.

Poziomy izolacji: Delta Lake domyślnie zapewnia poziom **WriteSerializable** lub **Serializable**. Czytający analityk zawsze widzi w 100% spójną migawkę danych (*Snapshot Isolation*) i nigdy nie blokuje piszącego!

Źródło: Delta Lake Protocol Documentation, "Optimistic Concurrency Control and Isolation Levels", 2023.

20. Apache Iceberg pod maską: Architektura drzewa metadanych

Apache Iceberg (stworzony przez Netflix, wspierany przez Snowflake, Apple i AWS) stosuje **3-poziomowe hierarchiczne drzewo metadanych**:



Genialna właściwość: Dzięki statystykom w plikach *Manifest List* i *Manifest Files* silnik SQL (Trino/Spark) może odrzucić 90% plików danych **przed wysłaniem jakiegokolwiek żądania do S3**! Planowanie zapytań trwa milisekundy nawet na petabajtach.

21. Ukryte partycjonowanie (Hidden Partitioning) w Iceberg

Najczęstszy błąd analityków w klasycznym jeziorze Hive: tabela jest partycjonowana po `event_date`, a analityk pisze filtr po znaczniku czasu `WHERE event_timestamp >= '2026-11-29 08:00:00'`:

Klasyczny koszmar (Hive / Stary Data Lake)

- Silnik nie wie, że `event_timestamp` wiąże się z partycją `event_date`.
- **Następuje pełne skanowanie całej tabeli (Full Table Scan)!**
- Zapytanie czyta 50 TB zamiast 50 GB.
- Analityk musiał pamiętać o dopisaniu sztucznego warunku: `AND event_date = '2026-11-29'`.
- Zmiana schematu partycjonowania (np. z dni na godziny) wymagała **przepisania 100% danych** w nowej strukturze katalogów.

Ukryte partycjonowanie (Apache Iceberg)

- Partycjonowanie jest **funkcją matematyczną** zdefiniowaną w metadanych: np. `day(event_timestamp)`.
- Użytkownik filtruje wyłącznie po `event_timestamp`.
- Iceberg **automatycznie wie**, które partycje odrzucić na poziomie metadanych!
- **Ewolucja partycjonowania (Partition Evolution):** Możesz zmienić partycjonowanie od dzisiaj na godziny – stare dane zostają w dniach, nowe w godzinach. Zero przepisywania historii!

Źródło: Apache Iceberg Documentation, "Partition Evolution & Hidden Partitioning", Apache Software Foundation.

22. Podróż w czasie (Time Travel) i audytowalność danych

Niemutowalność plików danych i numerowane wersje metadanych w Delta Lake / Iceberg umożliwiają funkcjonalność **Time Travel** (odczyt stanu tabeli z przeszłości):

```sql
-- Odpytanie tabeli Delta Lake o stan z konkretnej wersji commitu:
SELECT * FROM sales_lakehouse VERSION AS OF 42;

-- Odpytanie tabeli o stan z dokładnego momentu w przeszłości:
SELECT * FROM sales_lakehouse TIMESTAMP AS OF '2026-11-29 23:59:59';
```

1. Ochrona przed katastrofą

Natychmiastowy Rollback

- Junior developer puścił błędny skrypt z `DELETE` bez klauzuli `WHERE`?
- W tradycyjnym jeziorze: dramat i odzyskiwanie z backupu (dni).
- W Lakehouse: `RESTORE TABLE TO VERSION AS OF 41` (trwa 2 sekundy!).

2. Powtarzalność w Machine Learning

Deterministyczny trening

- Model wytrenowany 6 miesięcy temu zachowuje się dziwnie?
- Data Scientist może wytrenować model na **dokładnie tej samej migawce danych**, która istniała w dniu pierwotnego treningu.

3. Audytowalność i zgodność

Zgodność finansowa

- KNF / audytor pyta: *"Jakie dane widział zarząd w momencie generowania bilansu kwartalnego?"*.
- Możliwość bezdyskusyjnego odtworzenia raportu z tamtej sekundy.

Źródło: Databricks Technical Documentation, "Work with Delta Lake table history & Time Travel", 2023.

23. Modyfikacje danych: Copy-on-Write (CoW) vs Merge-on-Read (MoR)

Pliki Parquet są fizycznie niemutowalne (nie da się zmienić 1 bajtu w środku pliku). Jak zatem zrealizować operacje `UPDATE`, `DELETE` lub `MERGE INTO`?

Copy-on-Write (CoW)

Domyślny dla analityki wsadowej (Delta / Iceberg)

- Jeśli modyfikujemy 1 wiersz w 100-megabajtowym pliku Parquet:
  ➔ **Cały plik Parquet jest odczytywany i zapisywany na nowo** w wersji zaktualizowanej.
- **Koszt zapisu:** Wysoki (zjawisko *Write Amplification*).
- **Wydajność odczytu:** **Maksymalna (10/10)**. Czytnik czyta czyste, w pełni skompresowane pliki Parquet bez żadnego narzutu!
- Idealne dla: rzadkich aktualizacji i intensywnych zapytań analitycznych BI.

Merge-on-Read (MoR)

Zoptymalizowany pod streaming i CDC (Hudi / Iceberg v2)

- Modyfikowany wiersz nie przepisuje pliku Parquet!
- Zamiast tego dopisywany jest mały **plik różnicowy (Delete File / Delta Log w Avro)**.
- **Koszt zapisu:** **Minimalny i błyskawiczny**.
- **Wydajność odczytu:** Niższa. Czytnik musi w locie połączyć (*merge*) plik Parquet z plikiem usunięć.
- Wymaga asynchronicznego procesu kompakcji w tle.

Źródło: Apache Hudi & Iceberg Documentation, "Table Types: Copy-on-Write vs Merge-on-Read", 2023.

24. Higiena fizyczna jeziora danych: Compaction i Z-Ordering

Nawet najlepszy format tabelaryczny ulegnie degradacji, jeśli inżynierowie nie zadbają o higienę fizyczną storage'u:

1. Kompakcja (Compaction / OPTIMIZE)

Walka z problemem małych plików

- Strumienie danych (Streaming) tworzą tysiące plików po 100 KB.
- Polecenie `OPTIMIZE tabela` scala małe pliki w optymalne bloki **128 MB – 1 GB**.
- Drastyczna redukcja narzutu na transakcje HTTP i metadane.

2. Wielowymiarowy Z-Ordering

Krzywa Morton Space-Filling Curve

- Tradycyjne sortowanie działa tylko dla 1. kolumny.
- **Z-Order** układa dane w plikach z zachowaniem lokalności wielowymiarowej (np. `data` + `klient_id`).
- Zwiększa skuteczność omijania plików (Data Skipping) **nawet 10-krotnie** przy wielokryteriowych filtrach!

3. Czyszczenie śmieci (VACUUM)

Usuwanie starych plików fizycznych

- Pliki logicznie usunięte nadal leżą na S3 (dla Time Travel).
- Polecenie `VACUUM tabela RETAIN 7 DAYS` fizycznie usuwa pliki starsze niż 7 dni.
- Odzyskiwanie gigabajtów i redukcja kosztów chmury.

Nowość w branży: Databricks wprowadził technologię **Liquid Clustering**, która zastępuje sztywne partycjonowanie i Z-Ordering dynamicznym, adaptacyjnym klastrowaniem danych w locie bez konieczności kosztownego przepisywania tabeli.

Źródło: Databricks Documentation, "OPTIMIZE and Z-Order Clustering", 2023.

25. Złota macierz porównawcza formatów tabelarycznych

Cecha / Kryterium	Delta Lake (Linux Foundation / Databricks)	Apache Iceberg (Apache / Netflix, Apple)	Apache Hudi (Apache / Uber)
Pochodzenie i główny autor	Databricks (twórcy Apache Spark)	Netflix (Dan Weeks, Ryan Blue)	Uber (Vinoth Chandar)
Główna architektura metadanych	Log transakcyjny `_delta_log` (JSON + Checkpoint Parquet)	Hierarchiczne drzewo migawek (Avro Manifest Lists)	Log timeline + indeksy kluczy (Hoodie Timeline)
Główna siła projektowa	Bezkonkurencyjna integracja ze Spark, Databricks i PowerBI	Niezależność od silnika (Trino, Snowflake, Spark, DuckDB, BigQuery)	Streaming Ingestion, natywny CDC, szybkie operacje Upsert
Partycjonowanie	Klasyczne + Liquid Clustering	**Ukryte partycjonowanie (Hidden Partitioning) & Evolution**	Partycjonowanie fizyczne z dedykowanymi indeksami
Domyślna obsługa modyfikacji	Copy-on-Write (oraz Deletion Vectors)	Copy-on-Write oraz Merge-on-Read (v2)	Copy-on-Write oraz Merge-on-Read (wybitny MoR)
Katalog metadanych	Ścieżka katalogowa lub Unity Catalog	**Dowolny katalog (Iceberg REST Catalog, Nessie, Glue)**	Ścieżka systemowa lub Hive Metastore
Neutralność ekosystemowa	Wysoka (projekt Open Source pod egidą Linux Foundation)	**Bardzo wysoka (standard de facto chmur i hurtowni)**	Średnia (wysoka specjalizacja w CDC/streaming)

Źródło: Analiza komparatywna na podstawie oficjalnej dokumentacji projektów Delta Lake, Apache Iceberg i Apache Hudi, 2024

26. Studia decyzyjne dla inżynierów (Dyskusja na sali)

Scenariusz A: Wymóg RODO ("Prawo do bycia zapomnianym") w petabajtowym jeziorze
Firma ubezpieczeniowa posiada 500 TB danych telemetrycznych kierowców składowanych w tabeli Lakehouse. Dziennie wpływa około 50 żądań od klientów żądających natychmiastowego usunięcia ich historii przejazdów (`DELETE WHERE user_id = X`). Każde zapytanie CoW przepisuje dziesiątki gigabajtów danych, drenując budżet chmury.
👉 **Pytanie inżynierskie:** Jak rozwiążesz ten problem architektonicznie? Czy przełączysz tabelę na Merge-on-Read z plikami usunięć (Deletion Vectors), czy zastosujesz okresową kompakcję zbiorczą? Uzasadnij wybór.

Scenariusz B: Miliony urządzeń IoT wysyłają odczyty co 5 sekund
Sieć 200 000 inteligentnych liczników energii elektrycznej przesyła odczyty telemetryczne przez Kafkę do Lakehouse'a. Po 2 tygodniach zapytania analityczne zarządu drastycznie spowalniają. Audyt wykazuje obecność **15 milionów małych plików Parquet po 40 KB każdy**.
👉 **Pytanie inżynierskie:** Co zawiodło w projekcie potoku zapisu? Jakie mechanizmy higieny fizycznej wdrożysz, aby przywrócić zapytania do latencji subsekundowej?

27. Złote zasady architekta w warstwie składowania

1. **Nigdy nie pozwalaj na analitykę na surowych plikach CSV/JSON w chmurze:**
   - Format tekstowy marnuje pasmo, CPU i budżet chmurowy. Surowe pliki konwertuj do formatu kolumnowego w pierwszej minucie po ich pozyskaniu.
2. **Kompakcja małych plików to nie luksus, lecz obowiązek higieniczny:**
   - Miliony małych plików po 50 KB zabiją każdy silnik analityczny (Spark, Trino, Athena) przez narzut operacji na metadanych i połączeń sieciowych. Dąż do plików o rozmiarze **256 MB – 1 GB**.
3. **Pliki Parquet bez formatu tabelarycznego to proszenie się o awarię:**
   - Jeśli zapisujesz dane na chmurowym Object Storage (S3/ADLS/GCS), zawsze nakładaj warstwę transakcyjną (**Delta Lake** lub **Apache Iceberg**). Zyskujesz atomowość, brak wyścigów danych i darmowy Time Travel.
4. **Dobieraj technikę modyfikacji do częstotliwości mutacji:**
   - Rzadkie aktualizacje i intensywne czytanie BI? ➔ **Copy-on-Write (CoW)**.
   - Ciągły strumień modyfikacji CDC ze źródeł OLTP? ➔ **Merge-on-Read (MoR)** z asynchronicznym scalaniem w tle.

Podsumowanie Wykładu 3

- **Ewolucja nośników:** Od drogich i sztywnych hurtowni **DWH**, przez tanie lecz chaotyczne **Data Lake**, po zunifikowany **Lakehouse** łączący tani Object Storage z transakcjami ACID.
- **Fizyka i sprzęt:** Układ kolumnowy (**Column-oriented**) eliminuje wąskie gardło I/O, optymalizuje wykorzystanie linii pamięci podręcznej CPU (**L1/L2/L3 Cache**) oraz umożliwia wektoryzację instrukcji **SIMD**.
- **FinOps i Parquet:** Inteligentne metadane pliku Parquet (**Row Groups, Pages, Footer**) wraz z selekcją kolumn i omijaniem bloków (**Predicate Pushdown**) redukują koszty zapytań w chmurze nawet o **99%**.
- **Pamięć operacyjna: Apache Arrow** stanowi kolumnowy standard in-memory, eliminując koszt serializacji danych pomiędzy różnymi silnikami i językami (**Zero-Copy**).
- **Formaty tabelaryczne: Delta Lake** (log `_delta_log` i protokół OCC) oraz **Apache Iceberg** (drzewo manifestów i ukryte partycjonowanie) rozwiązują problem braku ACID, wyścigów danych i koszmaru operacji `LIST` na obiektowych magazynach S3.

Dziękuję za uwagę!

Następny wykład (W04):

Rozproszone przetwarzanie wsadowe: Apache Spark

Od MapReduce do przetwarzania w pamięci • Architektura klastra: Driver, Executors, Cluster Manager • DataFrame API • Diagnoza awarii: OOM, Data Skew i Shuffling

Specyfikacje, materiały źródłowe i diagramy dostępne w repozytorium kursu.